GET SHIT DONE · V1.2.0 + EVAL-FIRST LAYER

Ship real software with Claude Code, phase by phase

A spec-driven framework: locked eval contracts, orchestrated agents, and a git-verified work ledger — from a raw idea to a merged PR.

Eval-first contracts | Work ledger | Autonomous waves | 67 commands

WHAT THIS DECK COVERS

Four ideas, then two builds end to end

01
What GSD is
The mental model — gated phases, agents, and the artifacts that hold project truth.

02
How it works
The lifecycle: new-project → discuss → plan → execute → verify → ship.

03
Key capabilities
Eval-first, MVP/SPIDR, TDD, quality gates, AI phases, mobile scaffolds.

04
How to use it
The command surface and the everyday loop you actually type.

05
Example · SaaS
“TaskFlow” team app — one phase walked file by file.

06
Example · Payments
“PayLink” gateway — mobile app + backend + admin dashboard.

01

What GSD is

A mental model before the mechanics.

WHY STRUCTURE BEATS VIBES

AI writes code fast — and drifts fast

Unstructured AI coding

- Scope creeps mid-session; the goal quietly moves
- “Looks right” passes for done — no objective proof
- Context runs out and the thread loses the plot
- No audit trail: why was this built this way?

GSD’s answer

- Phases lock scope; gray areas resolved up front
- A **locked eval contract** proves done, deterministically
- A **work ledger** survives resets — resume exactly where you stopped
- Every decision & gate is a durable artifact in `.planning/`

IN ONE SENTENCE

GSD turns an idea into shipped code through gated phases

Each phase is a vertical slice of user value that is **discussed**, **planned**, **executed** and **verified** by specialized agents — and it cannot pass until its pre-committed evals go green.

> This install is **eval-gsd**: original GSD *plus* an additive eval-first + context-resilient layer. Turn the layer off in config and it behaves exactly like classic GSD. Same 67 commands, two modes.

THE CORE IDEA

Six principles that hold the whole thing together

Decompose

Projects break into vertical phases — each a complete, testable slice of value, not a horizontal layer.

Gate with evals

Every phase locks an eval contract before code. Deterministic rows become the executor's hard gate.

Orchestrate

Specialized subagents — planner, executor, verifier — run in dependency-ordered parallel waves.

Persist

A git-verified work ledger survives context resets, so any session resumes exactly where the last stopped.

Verify backward

Verification works back from the phase goal — task done ≠ goal achieved.

Audit

Every decision, commit and gate result is captured as a durable artifact under .planning/.

WHO DOES WHAT

You are the visionary; Claude is the builder

◉

You decide

- **What** to build and how it should behave
- Trade-offs at genuine decision points
- Sign-off on human-verify checkpoints

✦

Claude builds

- Technical **how**, feasibility, edge cases
- Plans, code, tests, commits, PRs
- Escalates only on a blocked gate or real choice

✓

Gates enforce

- Pre-flight, revision, escalation, abort
- Eval rows that must go green to pass
- Weakening & gaming detection at verify

ARTIFACT ECOSYSTEM

Project truth lives in `.planning/`

ARTIFACT	HOLDS	WRITTEN BY
PROJECT.md	Vision, core value, validated vs active requirements	new-project
REQUIREMENTS.md	REQ-01..N with acceptance criteria + traceability	new-project / discuss
ROADMAP.md	Phases with goals, dependencies, success criteria	roadmapper
NN-CONTEXT.md	Locked decisions, domain terms, code references	discuss-phase
NN-EVAL-CONTRACT.md	Code / Judge / Human rows + `locked_hash`	spec / discuss
NN-YY-PLAN.md	Typed tasks with `acceptance_criteria`	planner
NN-VERIFICATION.md	Must-haves verdict + eval verdict	verifier
LEDGER.md	Task records, evidence hashes, escalations	execute (orchestrator)

THE AGENTS

A team of specialists, not one prompt

AGENT	JOB
roadmapper	Requirements → phased roadmap
phase-researcher	Investigate unknowns before planning
planner	Phase scope → typed task plans
plan-checker	Validate plans; loop back to planner
executor	Run tasks, commit per task, hit gates
verifier	Goal-backward + eval verdict
code / security / ui	Review & audit specialists

- **Named contracts** — each agent has a defined input, output marker, and done-condition.
- **Wave parallelism** — plans group into W1→W2→W3 by dependency; same-wave plans run concurrently.
- **Worktree isolation** — executors run in their own git worktree; the main tree stays clean.
- **Fresh context** — each agent starts clean and reads only the artifacts it needs.

02

How it works

The lifecycle you run, stage by stage.

END TO END

One project, this loop per phase

INIT
new-project
Questioning → requirements → roadmap

→

PER PHASE
discuss
Lock decisions in CONTEXT.md

→

PER PHASE
plan
Planner ↔ checker loop

→

PER PHASE
execute
Parallel waves, hard gates

→

PER PHASE
verify
Goal-backward + eval verdict

→

CLOSE
ship
PR from artifacts

Autonomous mode drives discuss→plan→execute→verify across every remaining phase, escalating to you only on a blocked gate or a genuine decision.

NEW-PROJECT

From a fuzzy idea to a locked roadmap

STEP 1
Question
Vision, users, must-haves vs nice-to-haves

→

STEP 2
Research
Parallel agents probe unknowns

→

STEP 3
Requirements
REQ-01..N, each testable

→

STEP 4
Roadmap
Phases + success criteria

> Produces `PROJECT.md`, `REQUIREMENTS.md`, `ROADMAP.md`, `STATE.md` — committed together as the project's spine.

SLICING THE ROADMAP

Vertical slices, not horizontal layers

Horizontal (avoid)

- Build all of the DB, then all of the API, then all of the UI
- Nothing is demoable until the very end
- Integration risk piles up to the finish

Vertical (GSD default)

- Each phase cuts through every layer for **one** capability
- Phase 1 is a working, shippable thin slice
- Risk is retired continuously, phase by phase

DISCUSS-PHASE

Resolve the gray areas before planning

Claude surfaces gray areas

- “Auth: JWT in a cookie, or a bearer token?”
- “Errors: retry, or surface to the user?”
- “Layout: sidebar, or top nav?”

Scope guard · No creep

You answer → it locks them

- Decisions written to `NN-CONTEXT.md`
- Domain terms + canonical code references
- Deferred ideas parked so they’re not re-asked

> New scope? “That’s a new phase, not this one.”

EVAL-FIRST CONTRACT

Quality gates are locked before any code

eval-contract.md

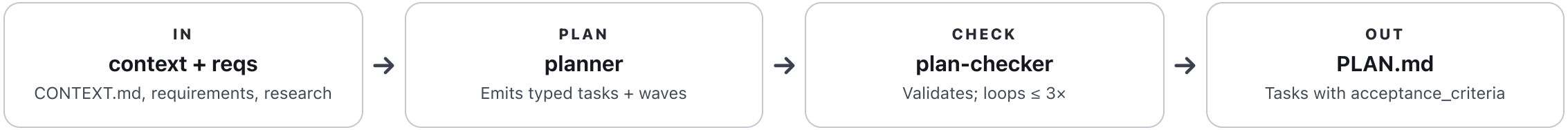
```
# NN-EVAL-CONTRACT.md   status: locked
| id   | req    | behavior          | measure | sev  |
| EC-1 | REQ-01 | login → 200 + cookie | Code    | gate |
| EC-2 | REQ-02 | bad creds → 401     | Code    | gate |
| EC-3 | REQ-03 | error copy is clear | Judge   | warn |
| EC-4 | REQ-01 | flow feels smooth   | Human   | warn |

locked_hash: sha256(normalized_rows)
```

- **Code** — deterministic CLI; the executor's hard gate (~80%).
- **Judge** — rubric scored by a model for subjective quality.
- **Human** — UAT / felt experience, batched for you.
- **Coverage gate** — every in-scope REQ needs ≥1 row; no orphans.

PLAN-PHASE

Scope becomes executable tasks



> Each `Code` + `gate` contract row becomes a task's `<acceptance_criteria>` — a CLI command the executor must run green.

EXECUTE-PHASE

Parallel waves, every task gated

- **Dependency analysis** — plans grouped into waves W1→W2→W3.
- **Parallel executors** — same-wave plans run at once, in isolated worktrees.
- **Hard gate per task** — run acceptance_criteria; exit ≠ 0 stops & escalates.
- **Commit per task** — traceable history; SUMMARY.md records hashes + self-check.
- **Ledger is truth** — orchestrator is the sole writer of LEDGER.md.

plan task — hard gate

```
<task type="auto">
  <name>POST /login issues a session</name>
  <acceptance_criteria>
    curl -si localhost:3000/login \
      -d '{"email":"a@b.co","pw":"x"}' \
      | grep -q 'Set-Cookie: sid='
  </acceptance_criteria>
</task>   # exit 0 → commit · else → halt
```

VERIFY-PHASE

Task done ≠ goal achieved

Goal-backward check

- Pull **must-haves** from the phase goal & success criteria
- Each truth: ✓ verified / ✕ failed / ? human
- Stubs and unwired code are caught here

Eval verdict

- **Coverage** — REQ ⇄ eval bijection holds
- **Weakening** — recompute `locked_hash`; must match
- **Gaming** — a Code row edited in its own commit is flagged
- **Gate rows green** — proven from ledger evidence

> Writes `NN-VERIFICATION.md` with status passed / failed / human_needed.

SHIP & AUTONOMOUS

Close the phase — or let the loop run

/gsd-ship

- Pre-flight: verification **passed**, tree clean, on a branch
- PR body built from goal + commits + eval verdict
- Optional `--review` spawns code-reviewer

/gsd-autonomous

- Loops every remaining phase hands-free
- Loop-control: detects non-convergence (S1/S2)
- Escalates & pauses instead of thrashing

GATES & CHECKPOINTS

Where the framework stops to think — or to ask you

GATE	FIRES WHEN
Pre-flight	A precondition is missing
Revision	Output quality is insufficient → loop
Escalation	A human decision is required
Abort	Continuing would cause damage

CHECKPOINT	MEANS
human-verify	“Look at it & confirm” (batched at phase end)
decision	“Choose A or B” — blocks until answered
human-action	“Enter the secret / API key”

THE WORK LEDGER

Why a dropped session is a non-event

- **Single source of truth** — every task: status, evidence hash, attempts, green-eval count.
- **Git-verified** — a “done” task is re-checked against its commit on resume.
- **Context-resilient** — at the context warning, the agent checkpoints the ledger; handoff is automatic.
- **Stall detection** — attempts spike → escalate, instead of looping forever.

ledger.md

```
# .planning/LEDGER.md  —  HEAD
| task     | phase    | status    | evidence |
|----------|----------|-----------|----------|
| 01-01-T1 | 01-auth  | COMPLETED | c36ae3c  |
| 01-01-T2 | 01-auth  | COMPLETED | a91f4d2  |
| 02-01-T1 | 02-board | IN_PROGRESS| —       |

resume → reads HEAD, git-verifies, continues
```

SHIPPING BOUNDARIES

Phases ship; milestones graduate

OPEN
new-milestone
“What’s next?” + planted seeds

→

RUN
phases
discuss→plan→execute→verify

→

AUDIT
audit-milestone
Coverage + cross-phase integration

→

CLOSE
complete-milestone
Archive roadmap; validate reqs

complete-milestone archives the roadmap and promotes shipped requirements into `PROJECT.md`’s *Validated* section — ready for the next cycle.

03

Key capabilities

The features that make it more than a checklist.

MEASUREMENT SPLIT

Prove ~80% by machine; reserve humans for what only they can judge

~80%

Code rows — deterministic CLI gates the executor runs every task

~15%

Judge rows — model-scored rubrics for subjective quality

~5%

Human rows — felt experience & UAT, batched at phase end

The split **drives architecture**: if a behavior can't be tested without a browser or device, factor the logic into a pure module so the ~80% stays machine-gatable.

MVP MODE + SPIDR

When a story is too big, split on one axis

AXIS	TRIGGER	SPLIT INTO
Spike	An unknown needs research first	Spike phase → implementation phase
Paths	Happy path + error paths	Happy path first → error cases
Interfaces	Web / mobile / API surfaces	One phase per interface
Data	Multiple data scopes	Small scope (one user) → larger
Rules	Many business rules	Basic rules first → complex policy

MVP mode opens with a **walking skeleton** — one task proving core value across every layer. Stories read `As a [role], I want [action], so that [value].`

TDD MODE

Behavior-adding tasks go red before green

RED
Write failing test
Commit the intent, watch it fail

→

GREEN
Make it pass
Smallest implementation that satisfies

→

REFACTOR
Clean up
Improve with the test as a net

Use for

- Business logic, validation, transforms
- Algorithms & API contracts

Skip for

- UI styling, config, glue code
- Exploratory prototyping

REVIEW ON DEMAND

Three specialist audits you can run any time

/gsd-code-review

Diff reviewed for bugs, simplification and risky patterns. Post inline (--comment) or auto-apply (--fix).

/gsd-secure-phase

STRIDE threat model vs the code: input validation, auth, crypto, secrets. Produces SECURITY.md.

/gsd-ui-review

Live UI audited against UI-SPEC.md across six visual pillars. Produces a scored UI-REVIEW.md.

AI-INTEGRATION-PHASE

A dedicated track for phases that call an LLM

1
Framework
Pick SDK / LangChain / …

→

2
AI-SPEC
Prompts, guardrails, cost

→

3
Eval plan
Quality, latency, cost rows

> Output is an `AI-SPEC.md`: framework decision, a prompt reference library, and an eval strategy so model quality is measured — not hoped for.

EVAL HOOKS

Mobile is ~80% gatable — here's how (React Native)

BEHAVIOR	COMMAND_OR_RUBRIC	SEVERITY
Types sound	`npx tsc --noEmit`	gate
Lint clean	`npx eslint . --max-warnings=0`	gate
Tests pass	`npm test -- --ci`	gate
JS bundles	`npx expo export --platform ios`	gate
Native build	`xcodebuild … / gradlew assembleDebug`	warn*
E2E on sim	`maestro test .maestro/`	warn*
Feels right	manual UAT	Human

* warn = gate-skips when the SDK / simulator is absent — and the skip is logged, never silently "covered".

04

How to use it

The commands you actually type.

THE SURFACE, GROUPED

67 commands — the dozen you'll live in

Lifecycle

- `/gsd-new-project`
- `/gsd-new-milestone`
- `/gsd-ship`
- `/gsd-complete-milestone`

Per phase

- `/gsd-discuss-phase N`
- `/gsd-spec-phase N`
- `/gsd-plan-phase N`
- `/gsd-execute-phase N`
- `/gsd-verify-phase N`

Quality & auto

- `/gsd-autonomous`
- `/gsd-code-review`
- `/gsd-secure-phase N`
- `/gsd-ui-review`
- `/gsd-ai-integration-phase`

THE EVERYDAY LOOP

Three commands carry most days

/gsd-progress

The situational command. Reads STATE + ledger, tells you exactly what to do next, and routes there.

/gsd-next

Advance to the next action or phase without thinking about which command comes next.

/gsd-resume-work

Reads LEDGER HEAD, git-verifies done work, halts on open escalations — full context restored.

Lost? `/gsd-progress` always knows where you are and what's next. `/gsd-health` and `/gsd-stats` diagnose the project at a glance.

ZERO TO SHIPPED

A whole project in a handful of commands

terminal

```
# 1 · scope it — questioning → requirements → roadmap
/gsd-new-project

# 2 · per phase: lock decisions, plan, build, verify
/gsd-discuss-phase 1
/gsd-plan-phase 1
/gsd-execute-phase 1     # waves + hard gates + verify

# 3 · or let it run the rest hands-free
/gsd-autonomous          # escalates only when it must

# 4 · close it out
/gsd-ship 1 --review     # PR + code review
```

05

Example 1 · SaaS app

“TaskFlow” — a team task-management product.

MEET TASKFLOW

A multi-tenant team task app

Workspaces with teams, a task board, real-time collaboration, and Stripe subscription billing. We'll scope it, then walk **Phase 1 (Auth)** file by file — the same loop every phase follows.

Constraints locked at new-project

- Multi-tenant: a user belongs to many workspaces
- Postgres + a Node/TS API + React SPA
- Stripe for subscriptions
- Ship thin slices weekly

NEW-PROJECT OUTPUT

Vision and testable requirements

PROJECT.md

```
# PROJECT.md
Core value: a team plans & tracks work
  in shared workspaces, in real time.

Active requirements:
  REQ-01  email + password auth
  REQ-02  workspaces & membership
  REQ-03  tasks: create / assign / move
  REQ-04  realtime board updates
  REQ-05  Stripe subscription billing
```

REQUIREMENTS.md

```
# REQUIREMENTS.md — each one testable
REQ-01 Auth
  AC: POST /login with valid creds
      → 200 + HttpOnly session cookie
  AC: invalid creds → 401, no cookie
  AC: error copy never says which
      field was wrong
  phase: 01
```

THE ROADMAP

Five vertical slices, each shippable

PHASE	GOAL	REQS	SHIPS
01 · Auth	Email/password login + sessions	REQ-01	Users can sign in
02 · Workspaces	Create workspace, invite members	REQ-02	Teams exist
03 · Board	Create / assign / move tasks	REQ-03	A usable task board
04 · Realtime	Live board updates over WebSocket	REQ-04	Collaboration
05 · Billing	Stripe subscriptions + webhooks	REQ-05	Revenue

Dependencies: 02→01, 03→02, 04→03, 05→02. The executor reads these and waves the work; 01 is a complete, demoable slice on its own.

DISCUSS → CONTEXT

Gray areas in, locked decisions out

Claude asks

- Session: JWT-in-cookie or server session?
- Password hashing: bcrypt or argon2?
- Lockout after N failed attempts?

You decide → NN-CONTEXT.md

- **HttpOnly cookie** holding a rotating session id
- **argon2id**, per-user salt
- Lockout deferred to the hardening phase

> “Add SSO?” → parked in *Deferred*, not built now. The phase scope stays exactly REQ-01.

THE EVAL CONTRACT

Locked before a single line of auth code

01-EVAL-CONTRACT.md

```
# 01-EVAL-CONTRACT.md          status: locked
| id   | req    | behavior                          | measure | sev  |
| EC-1 | REQ-01 | valid login → 200 + Set-Cookie    | Code    | gate |
| EC-2 | REQ-01 | bad creds → 401, no cookie        | Code    | gate |
| EC-3 | REQ-01 | password stored argon2id, never plaintext | Code | gate |
| EC-4 | REQ-01 | error copy never reveals which field failed | Judge | warn |
| EC-5 | REQ-01 | sign-in flow feels smooth on mobile web     | Human | warn |

coverage: REQ-01 → 5 rows (clean)     locked_hash: 9f2a…c7
```

3 Code gates | 1 Judge | 1 Human

PLAN → EXECUTE → SUMMARY

The contract becomes runnable tasks

01-01-PLAN.md planner	**T1** hash+store user · **T2** POST /login · **T3** session cookie middleware
T2 acceptance hard gate	`curl -si .../login -d @ok.json | grep -q 'Set-Cookie: sid='` → EC-1
T1 acceptance hard gate	`node test/hash.mjs` asserts stored value starts `$argon2id$` → EC-3
01-01-SUMMARY.md executor	Commits `c36ae3c`, `a91f4d2`, `7b1e0aa` · self-check PASSED

EC-2 also runs green. EC-4 (Judge) is scored by a model; EC-5 (Human) is queued to the phase-end UAT batch.

VERIFY & SHIP

Proven, not assumed — then a PR

01-VERIFICATION.md

```
# 01-VERIFICATION.md
status: passed
eval_verdict:
  coverage:  clean
  weakening: clean  (hash matches)
  gaming:    none
  gate_rows: 3/3 green

| must-have            | result |
| login issues session | ✓ PASS |
| secrets never stored | ✓ PASS |
```

- **/gsd-ship 1 --review**
- PR body auto-built: goal, the 3 commits, eval verdict, UAT note
- code-reviewer scans the diff before merge
- Phase 1 shipped — the loop repeats for 02–05

SCALING OUT

The same loop, four more times

PHASE	NOTABLE EVAL ROWS	MIX
02 Workspaces	membership query returns only my workspaces (Code)	Code-heavy
03 Board	move task persists column + order (Code); board reads clean (Judge)	Code + Judge
04 Realtime	2 clients converge on edit (Code, headless); latency feels live (Human)	Code + Human
05 Billing	Stripe webhook is idempotent (Code); failed-card UX (Judge); receipt (Human)	All three

complete-milestone archives the roadmap and promotes REQ-01..05 to *Validated*. TaskFlow v1 is done; v2 starts with `/gsd-new-milestone`.

06

Example 2 · Payment gateway

“PayLink” — merchant mobile app + backend + admin dashboard.

MEET PAYLINK

One product, three surfaces, one shared backbone

Merchant app

React Native. Onboard, accept a payment, watch balance & payouts, get push receipts.

Backend / API

Node + TS. Accounts, charge creation, idempotent webhooks, payout ledger — the dependency for everything.

Admin dashboard

Web. Merchant management, transaction monitoring, disputes, metrics — behind RBAC.

Money + three clients + compliance = exactly where eval-first earns its keep. We'll see the **SPIDR Interfaces** split, real **mobile eval hooks**, a **STRIDE** pass, and an AI phase.

REQUIREMENTS ACROSS SURFACES

Numbered once, traced everywhere

SURFACE	REQUIREMENTS
Backend	REQ-01 merchant auth · REQ-02 create charge · REQ-03 idempotent webhook · REQ-04 payout ledger
Merchant app	REQ-05 onboarding · REQ-06 accept payment (QR) · REQ-07 balance & payouts · REQ-08 push receipts
Admin	REQ-09 merchant management · REQ-10 txn monitoring · REQ-11 disputes · REQ-12 metrics (RBAC)
Cross-cutting	REQ-13 fraud-risk score · REQ-14 PCI-aligned hardening · REQ-15 rate limits

SPIDR · INTERFACES SPLIT

“Accept a payment” spans three clients → three phases

One story, too big:
“As a merchant, I want to take a card payment and see it land in my balance, so that I get paid.”

DEPENDS
Backend
charge + webhook is the contract everyone calls

→

THEN
Mobile
merchant initiates & sees the result

→

THEN
Admin
staff monitor & resolve

Touches API + mobile + admin and crosses the happy/error paths. SPIDR splits it on the **Interfaces** axis first, then **Paths** within each.

ROADMAP ACROSS SURFACES

Backend first; mobile and admin parallelize as workstreams

PHASE	SURFACE	GOAL
01	Backend	Merchant auth + accounts API
02	Backend	Create charge + **idempotent** webhook
03	Backend	Payout ledger (double-entry)
04	Mobile	Onboarding + accept payment (RN)
05	Mobile	Balance, payout history, push
06	Admin	Merchant management + RBAC
07	Admin	Txn monitoring + disputes + metrics
08	AI	Fraud-risk scoring assist
09	All	PCI-aligned hardening + rate limits

Phases 04 and 06 both depend only on the backend — run them as parallel `--ws` workstreams.

BACKEND: THE MEATY GATE

Idempotency is a deterministic Code gate

02-EVAL-CONTRACT.md

```
# 02-EVAL-CONTRACT.md          status: locked
| id   | req    | behavior                   | measure |
| EC-1 | REQ-02 | POST /charges → 201 + id   | Code    |
| EC-2 | REQ-03 | replayed webhook applies   | Code    |
|      |        | exactly once               |         |
| EC-3 | REQ-03 | bad signature → 401        | Code    |
| EC-4 | REQ-02 | decline UX is clear        | Judge   |
```

hard gate — exactly-once

```
# the EC-2 gate (acceptance_criteria)
node test/webhook-dedupe.mjs
  # posts same event twice with one
  # Idempotency-Key; asserts the
  # ledger moved balance ONCE
→ exit 0  ✓ EC-2 green
```

Authoring the contract first forces the design: an **Idempotency-Key** table and a double-entry ledger — because that's the only way EC-2 can go green.

SECURE-PHASE · STRIDE

Threat-model the charge endpoint before shipping

STRIDE	THREAT	MITIGATION (VERIFIED IN CODE)
Tampering	Forged webhook body	HMAC signature check → EC-3 gate
Repudiation	“I never got paid”	Append-only ledger + evidence hashes
Info disclosure	PAN in logs	No card data stored; tokens only (PCI)
DoS	Charge-spam a merchant	Per-key rate limit → REQ-15
Elevation	Merchant A reads B’s charges	Tenant scoping on every query

/gsd-secure-phase writes SECURITY.md and verifies each mitigation actually exists in the diff — not just claimed.

MOBILE: REAL EVAL HOOKS

React Native is ~80% gatable — the contract proves it

BEHAVIOR (MERCHANT APP)	COMMAND_OR_RUBRIC	SEV
Types & lint sound	`npx tsc --noEmit · eslint`	gate
Charge-flow logic unit-tested	`npm test -- --ci`	gate
App bundles for iOS	`npx expo export --platform ios`	gate
Accept-payment e2e on sim	`maestro test .maestro/pay.yaml`	warn*
Tap-to-pay *feels* instant	manual UAT on a real device	Human

> Logic is factored **out of the components** (the `call-core.js` discipline), so the charge state machine is unit-gated without a render surface.

MOBILE: DECISIONS & SPLIT

discuss locks the platform questions; the mix stays honest

CONTEXT.md decisions

- Expo managed RN (no custom native module needed)
- QR-presented charge; card entry stays server-side (PCI)
- Optimistic UI, reconciled by webhook

Measurement mix

- **Code** gates: types, tests, bundle — always run
- **warn**: sim e2e & device build — skip+log if no SDK
- **Human**: gesture feel, push-receipt UX
- Store submission → Human, never a Code gate

ADMIN: RBAC + UI REVIEW

Read-only first; privileged actions split out by Paths

- **RBAC gate (Code)** — a support role hitting a refund route gets 403.
- **Tenant isolation (Code)** — admin queries scope to permitted merchants only.
- **Metrics correctness (Code)** — daily volume = sum of settled charges, asserted.
- **/gsd-ui-review** — dashboard audited vs UI-SPEC.md across six visual pillars.

Paths split

- P06: view & manage merchants (read)
- P07: disputes & refunds (privileged write)
- Risky writes get their own gates + human-verify

AI: FRAUD-RISK ASSIST

An LLM phase gets its own spec and evals

1
Framework
Claude SDK + tool use

→

2
AI-SPEC
Prompt, guardrails, cost cap

→

3
Eval plan
Labeled set + thresholds

Code/Judge evals

- Precision/recall vs a labeled charge set ≥ target
- p95 latency < budget; cost/call < cap
- Never auto-blocks — only *flags* for review

Human evals

- Analyst agrees the flag reasons are sensible
- False-positive friction is acceptable

AUTONOMOUS + INTEGRATION

Run the surfaces, then prove they connect

- **/gsd-autonomous --from 2** — drives backend, mobile and admin phases; pauses only on decisions or a blocked gate.
- **Loop-control** — if an eval won't converge (S1 retries / S2 negative progress), it escalates instead of thrashing.
- **audit-milestone** — integration-checker verifies the e2e flow: app charge → backend → webhook → admin sees it.

integration check

```
# cross-surface e2e (warn smoke)
mobile: maestro pay.yaml  → charge id
backend: replay webhook   → ledger +1
admin: GET /txns/<id>     → settled
✓ the slice works end-to-end
```

complete-milestone promotes REQ-01..15 to *Validated*. PayLink v1 ships with a proof trail for every surface — exactly what an auditor (or a future you) needs.

07

Wrap-up

When to reach for it, and how to start.

WHEN GSD SHINES — AND WHEN IT DOESN'T

Match the tool to the job

Overkill for

- A one-off script or throwaway spike
- A tiny fix with no real spec
- Pure exploration where the goal is still unknown

Built for

- Multi-phase products with real requirements
- Teams that want autonomy **and** governance
- Anything where "prove it works" matters — payments, data, AI

Need classic-GSD speed? Set `eval_first.require_contract: false` and the gating layer no-ops.

FOUR THINGS TO REMEMBER

If you keep only this

01

Gated phases

Small vertical slices, each with locked scope and a pre-committed definition of done.

02

Eval-first

~80% proven by deterministic Code gates; humans judge only what only they can.

03

Orchestrated

Specialist agents run in parallel waves; you decide, gates enforce, the loop drives.

04

Resumable

A git-verified ledger means a dropped context is a non-event — resume and continue.

YOUR FIRST HOUR

Three commands and you’re moving

get started

```
# point GSD at your idea
/gsd-new-project

# build the first slice, end to end
/gsd-discuss-phase 1
/gsd-plan-phase 1
/gsd-execute-phase 1

# lost? this always knows what’s next
/gsd-progress
```

- **Explore the install** — references/ explains every concept in depth.
- **Real examples** — the slugify, WebRTC and dogfood projects show full .planning/ trails.
- **Tune it** — /gsd-config toggles eval-first, TDD, MVP and model profiles.
- **Help** — /gsd-help lists all 67 commands with usage.

Get shit done — from idea to merged PR, with proof at every gate.